

Range

Static Range Sum Queries

```
#include

using namespace std;
typedef long long ll;
const int maxN = 2e5;

int N, Q, a, b;
ll x[maxN+1];

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 1; i <= N; i++){
        scanf("%lld", &x[i]);
        x[i] += x[i-1];
    }
    for(int q = 0; q < Q; q++){
        scanf("%d %d", &a, &b);
        printf("%lld\n", x[b]-x[a-1]);
    }
}
```

Static Range Minimum Queries

```
#include

using namespace std;
const int maxN = 2e5+1;
const int logN = 19;

int N, Q, a, b, l, k, x[maxN][logN];

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 0; i < N; i++)
        scanf("%d", &x[i][0]);

    for(int i = 1; i < logN; i++)
        for(int j = 0; j <= N-(1<
```

Dynamic Range Sum Queries

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5;

int N, Q, t, a, b;
ll x[maxN+1], ds[maxN+1];

void update(int idx, ll val){
    ll diff = val - x[idx];
    for(int i = idx; i <= N; i += -i&i)
        ds[i] += diff;
    x[idx] = val;
}

ll query(int idx){
    ll sum = 0;
    for(int i = idx; i > 0; i -= -i&i)
        sum += ds[i];
    return sum;
}

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 1; i <= N; i++){
        scanf("%lld", &x[0]);
        update(i, x[0]);
    }
    for(int q = 0; q < Q; q++){
        scanf("%d %d %d", &t, &a, &b);
        if(t == 1) update(a, b);
        else printf("%lld\n", query(b)-query(a-1));
    }
}

```

Dynamic Range Minimum Queries

```

#include

using namespace std;
const int maxN = 2e5, SIZE = 4*maxN;
const int INF = 0x3f3f3f3f;

int N, Q, t, a, b, lo[SIZE], hi[SIZE], mn[SIZE];

void push(int i){
}

void pull(int i){
    mn[i] = min(mn[2*i], mn[2*i+1]);
}

void init(int i, int l, int r){
    lo[i] = l; hi[i] = r;
    if(l == r){
        scanf("%d", &mn[i]);
        return;
    }
}

```

```

    int m = 1+(r-l)/2;
    init(2*i, l, m);
    init(2*i+1, m+1, r);
    pull(i);
}

void assign(int i, int l, int r, int v){
    if(l > hi[i] || r < lo[i]) return;
    if(l <= lo[i] && hi[i] <= r){
        mn[i] = v; return;
    }

    push(i);
    assign(2*i, l, r, v);
    assign(2*i+1, l, r, v);
    pull(i);
}

int minimum(int i, int l, int r){
    if(l > hi[i] || r < lo[i]) return INF;
    if(l <= lo[i] && hi[i] <= r) return mn[i];

    push(i);
    int lmin = minimum(2*i, l, r);
    int rmin = minimum(2*i+1, l, r);
    pull(i);

    return min(lmin, rmin);
}

int main(){
    scanf("%d %d", &N, &Q);

    init(1, 1, N);
    for(int q = 0; q < Q; q++){
        scanf("%d %d %d", &t, &a, &b);
        if(t == 1) assign(1, a, a, b);
        else if(t == 2) printf("%d\n", minimum(1, a, b));
    }
}

```

Range Xor Queries

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5;

int N, Q, a, b;
ll x[maxN+1];

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 1; i <= N; i++){
        scanf("%lld", &x[i]);
        x[i] ^= x[i-1];
    }
    for(int q = 0; q < Q; q++){
        scanf("%d %d", &a, &b);
        printf("%lld\n", x[b]^x[a-1]);
    }
}

```

```
    }  
}
```

Range Update Queries

```
#include  
  
using namespace std;  
typedef long long ll;  
const int maxN = 2e5+1;  
  
int N, Q, t, a, b, k;  
ll x, ds[maxN];  
  
void update(int idx, ll val){  
    for(int i = idx; i <= N; i += -i&i)  
        ds[i] += val;  
}  
  
ll query(int idx){  
    ll sum = 0;  
    for(int i = idx; i > 0; i -= -i&i)  
        sum += ds[i];  
    return sum;  
}  
  
int main(){  
    scanf("%d %d", &N, &Q);  
    for(int i = 1; i <= N; i++){  
        scanf("%lld", &x);  
        update(i, x);  
        update(i+1, -x);  
    }  
    for(int q = 0; q < Q; q++){  
        scanf("%d", &t);  
        if(t == 1){  
            scanf("%d %d %lld", &a, &b, &x);  
            update(b+1, -x);  
            update(a, x);  
        } else if(t == 2){  
            scanf("%d", &k);  
            printf("%lld\n", query(k));  
        }  
    }  
}
```

Forest Queries

```
#include  
  
using namespace std;  
const int maxN = 1001;
```

```

char c;
int N, Q, a[maxN][maxN];

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 1; i <= N; i++){
        for(int j = 1; j <= N; j++){
            scanf(" %c", &c);
            a[i][j] = a[i-1][j]+a[i][j-1]-a[i-1][j-1];
            if(c == '*') a[i][j]++;
        }
    }
    for(int q = 0; q < Q; q++){
        int x1, y1, x2, y2;
        scanf("%d %d %d %d", &x1, &y1, &x2, &y2);
        printf("%d\n", a[x2][y2]-a[x2][y1-1]-a[x1-1][y2]+a[x1-1][y1-1]);
    }
}

```

Hotel Queries

```

#include

using namespace std;
const int maxN = 2e5+1;

int N, M, h[maxN], k, ans;
int lo[4*maxN], hi[4*maxN], mx[4*maxN], mp[maxN];

void pull(int i){
    mx[i] = (h[mx[2*i]] >= h[mx[2*i+1]] ? mx[2*i] : mx[2*i+1]);
}

void init(int i, int l, int r){
    lo[i] = l; hi[i] = r;
    if(l == r){
        mp[l] = i;
        mx[i] = l;
        return;
    }
    int m = l+(r-l)/2;
    init(2*i, l, m);
    init(2*i+1, m+1, r);
    pull(i);
}

void update(int i, int v){
    h[i] -= v;
    i = mp[i];
    i >>= 1;
    while(i > 0){
        pull(i);
        i >>= 1;
    }
}

int find(int i, int v){
    if(lo[i] == hi[i]) return lo[i];
    return (h[mx[2*i]] >= v ? find(2*i, v) : find(2*i+1, v));
}

```

```

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 1; i <= N; i++){
        scanf("%d", &h[i]);
        init(1, 1, N);

        for(int i = 0; i < M; i++){
            scanf("%d", &k);
            ans = h[mx[1]] < k ? 0 : find(1, k);
            if(ans) update(ans, k);
            printf("%d%c", ans, (" \n")[i==M-1]);
        }
    }
}

```

List Removals

```

#include
#include
#include

using namespace std;
using namespace __gnu_pbds;
const int maxN = 2e5;

int N, p, x[maxN+1];
tree, rb_tree_tag,
tree_order_statistics_node_update> T;

int main(){
    scanf("%d", &N);
    for(int i = 1; i <= N; i++){
        scanf("%d", &x[i]);
        T.insert(i);
    }

    for(int i = 0; i < N; i++){
        scanf("%d", &p);
        printf("%d%c", x[*T.find_by_order(p-1)], (" \n")[i==N-1]);
        T.erase(T.find_by_order(p-1));
    }
}

```

Salary Queries

```

#include
#include
#include

using namespace std;
using namespace __gnu_pbds;
typedef pair pii;

```

```

const int maxN = 2e5;

char c;
int N, Q, a, b, p[maxN+1];
tree, rb_tree_tag,
tree_order_statistics_node_update> T;

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 1; i <= N; i++){
        scanf("%d", &p[i]);
        T.insert({p[i], i});
    }

    for(int i = 0; i < Q; i++){
        scanf(" %c %d %d", &c, &a, &b);
        if(c == '!'){
            T.erase({p[a], a});
            T.insert({b, a});
            p[a] = b;
        } else if(c == '?'){
            printf("%ld\n", T.order_of_key({b+1, 0}) - T.order_of_key({a, 0}));
        }
    }
}

```

Prefix Sum Queries

```

#include

using namespace std;
typedef long long ll;
typedef pair pll;
const int maxN = 2e5+1;
const int SIZE = 4*maxN;

int N, Q, lo[SIZE], hi[SIZE], mp[maxN];
ll sum[SIZE], pre[SIZE];

void pull(int i){
    pre[i] = max(pre[2*i], sum[2*i]+pre[2*i+1]);
    sum[i] = sum[2*i] + sum[2*i+1];
}

void init(int i, int l, int r){
    lo[i] = l; hi[i] = r;
    if(l == r){
        scanf("%lld", &sum[i]);
        pre[i] = max(0LL, sum[i]);
        mp[l] = i;
        return;
    }
    int m = (l+r)/2;
    init(2*i, l, m);
    init(2*i+1, m+1, r);
    pull(i);
}

void update(int idx, int val){
    int i = mp[idx];
    sum[i] = val;
}

```

```

pre[i] = max(0LL, sum[i]);

i >>= 1;
while(i){
    pull(i);
    i >>= 1;
}

pll query(int i, int l, int r){
    if(l > hi[i] || r < lo[i])    return {0, 0};
    if(l <= lo[i] && hi[i] <= r) return {pre[i], sum[i]};

    pll left = query(2*i, l, r);
    pll right = query(2*i+1, l, r);
    return {max(left.first, left.second+right.first), left.second+right.second};
}

int main(){
    scanf("%d %d", &N, &Q);
    init(1, 1, N);
    for(int q = 0, t, a, b; q < Q; q++){
        scanf("%d %d %d", &t, &a, &b);
        if(t == 1)
            update(a, b);
        else if(t == 2)
            printf("%lld\n", query(1, a, b).first);
    }
}

```

Pizzeria Queries

```

#include

using namespace std;
const int maxN = 2e5+1, SIZE = 4*maxN;
const int INF = 0x3f3f3f3f;

int N, Q, p[maxN], mp[maxN], lo[SIZE], hi[SIZE], mn[2][SIZE];

void pull(int i){
    mn[0][i] = min(mn[0][2*i], mn[0][2*i+1]);
    mn[1][i] = min(mn[1][2*i], mn[1][2*i+1]);
}

void init(int i, int l, int r){
    lo[i] = l; hi[i] = r;
    if(l == r){
        scanf("%d", &p[l]);
        mn[0][i] = l+p[l];
        mn[1][i] = (N-l+1)+p[l];
        mp[l] = i;
        return;
    }
    int m = (l+r)/2;
    init(2*i, l, m);
    init(2*i+1, m+1, r);
    pull(i);
}

void update(int idx, int val){

```

```

    int i = mp[idx];
    mn[0][i] = idx+val;
    mn[1][i] = (N-idx+1)+val;
    i >>= 1;
    while(i){
        pull(i);
        i >>= 1;
    }
}

int query(int i, int t, int l, int r){
    if(l > hi[i] || r < lo[i])    return INF;
    if(l <= lo[i] && hi[i] <= r)    return mn[t][i];

    int left = query(2*i, t, l, r);
    int right = query(2*i+1, t, l, r);
    return min(left, right);
}

int main(){
    scanf("%d %d", &N, &Q);
    init(1, 1, N);
    for(int q = 0, t, k, x; q < Q; q++){
        scanf("%d %d", &t, &k);
        if(t == 1){
            scanf("%d", &x);
            update(k, x);
        } else if(t == 2){
            int left = query(1, 1, 1, k);
            int right = query(1, 0, k, N);
            printf("%d\n", min(left-(N-k+1), right-k));
        }
    }
}

```

Range Interval Queries

```

#include

using namespace std;
typedef array operation;
const int maxN = 2e5+5;

int N, Q, ds[maxN], ans[maxN];
vector ops;

void update(int idx, int val){
    for(int i = idx; i <= N; i += -i&i)
        ds[i] += val;
}

int query(int idx){
    int sum = 0;
    for(int i = idx; i > 0; i -= -i&i)
        sum += ds[i];
    return sum;
}

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 1, x; i <= N; i++){

```

```

scanf("%d", &x);
ops.push_back({x, 0, i, i, -1});
}
for(int q = 1, a, b, c, d; q <= Q; q++){
    scanf("%d %d %d %d", &a, &b, &c, &d);
    ops.push_back({c, -1, a, b, q});
    ops.push_back({d, 1, a, b, q});
}

sort(ops.begin(), ops.end());
for(operation op : ops){
    int type = op[1];
    if(type == 0)    update(op[2], 1);
    else {
        int a = op[2], b = op[3], qid = op[4];
        int cnt = query(b) - query(a-1);
        ans[qid] += type * cnt;
    }
}

for(int q = 1; q <= Q; q++)
    printf("%d\n", ans[q]);
}

```

Subarray Sum Queries

```

#include

using namespace std;
typedef long long ll;
const int SIZE = 8e5;

int N, Q, k, lo[SIZE], hi[SIZE], mp[SIZE];
ll x, sum[SIZE], dp[SIZE], dpl[SIZE], dpr[SIZE];

void pull(int i){
    dpl[i] = max(dpl[2*i], sum[2*i] + dpl[2*i+1]);
    dpr[i] = max(dpr[2*i+1], sum[2*i+1] + dpr[2*i]);
    dp[i] = max(dpr[2*i] + dpl[2*i+1], max(dp[2*i], dp[2*i+1]));
    sum[i] = sum[2*i] + sum[2*i+1];
}

void build(int i, int l, int r){
    lo[i] = l; hi[i] = r;
    if(l == r){
        scanf("%lld", &sum[i]);
        dpl[i] = dpr[i] = dp[i] = sum[i];
        mp[l] = i;
        return;
    }
    int m = l+(r-l)/2;
    build(2*i, l, m);
    build(2*i+1, m+1, r);
    pull(i);
}

void update(int a, ll b){
    a = mp[a];

    dpl[a] = dpr[a] = dp[a] = sum[a] = b;
    a >= 1;
}

```

```

    while(a > 0){
        pull(a);
        a >>= 1;
    }
}

int main(){
    scanf("%d %d", &N, &Q);

    build(1, 1, N);
    for(int q = 0; q < Q; q++){
        scanf("%d %lld", &k, &x);
        update(k, x);
        printf("%lld\n", max(0LL, dp[1]));
    }
}

```

Subarray Sum Queries II

```

#include

using namespace std;
typedef long long ll;
typedef array result;
const ll INF = 0x3f3f3f3f3f3f3f;
const int SIZE = 8e5;

int N, Q, lo[SIZE], hi[SIZE];
ll sum[SIZE], dp[SIZE], dpl[SIZE], dpr[SIZE];

result operator+(const result &a, const result &b){
    return {
        a[0] + b[0],
        max(a[1], a[0] + b[1]),
        max(b[2], b[0] + a[2]),
        max(max(a[3], b[3]), a[2] + b[1]),
    };
}

void pull(int i){
    dpl[i] = max(dpl[2*i], sum[2*i] + dpl[2*i+1]);
    dpr[i] = max(dpr[2*i+1], sum[2*i+1] + dpr[2*i]);
    dp[i] = max(dpr[2*i] + dpl[2*i+1], max(dp[2*i], dp[2*i+1]));
    sum[i] = sum[2*i] + sum[2*i+1];
}

void build(int i, int l, int r){
    lo[i] = l; hi[i] = r;
    if(l == r){
        scanf("%lld", &sum[i]);
        dpl[i] = dpr[i] = dp[i] = sum[i];
        return;
    }
    int m = l+(r-l)/2;
    build(2*i, l, m);
    build(2*i+1, m+1, r);
    pull(i);
}

result query(int i, int l, int r){
    if(lo[i] > r || hi[i] < l) return {-INF, -INF, -INF, -INF};

```

```

    if(lo[i] >= l && hi[i] <= r) return {sum[i], dpl[i], dpr[i], dp[i]};
    result left = query(2*i, l, r);
    result right = query(2*i+1, l, r);
    return left + right;
}

int main(){
    scanf("%d %d", &N, &Q);

    build(1, 1, N);
    for(int q = 0, l, r; q < Q; q++){
        scanf("%d %d", &l, &r);
        result res = query(1, l, r);
        printf("%lld\n", res[3] <= 0 ? 0 : res[3]);
    }
}

```

Distinct Values Queries

```

#include

using namespace std;
typedef array triple;
const int maxN = 2e5+1;

int N, Q, a, b, x[maxN], ptr[maxN], ds[maxN], ans[maxN];
triple queries[maxN];
map mp;
set S;

void update(int idx, int val){
    for(int i = idx; i <= N; i += -i&i)
        ds[i] += val;
}

int query(int idx){
    int sum = 0;
    for(int i = idx; i > 0; i -= -i&i)
        sum += ds[i];
    return sum;
}

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 1; i <= N; i++){
        scanf("%d", &x[i]);

        for(int i = N; i > 0; i--){
            if(mp[x[i]])
                ptr[i] = mp[x[i]];
            mp[x[i]] = i;
        }

        for(int i = 1; i <= N; i++){
            if(S.count(x[i]) == 0)
                update(i, 1);
            S.insert(x[i]);
        }

        for(int i = 0; i < Q; i++){
            scanf("%d %d", &a, &b);

```

```

        queries[i] = {a, b, i};
    }
    sort(queries, queries+Q);

    int l = 1;
    for(int q = 0; q < Q; q++){
        while(l < queries[q][0]){
            if(ptr[l])
                update(ptr[l], 1);
            l++;
        }
        ans[queries[q][2]] = query(queries[q][1]) - query(queries[q][0]-1);
    }

    for(int q = 0; q < Q; q++)
        printf("%d\n", ans[q]);
}

```

Distinct Values Queries II

```

#include

using namespace std;
typedef pair pii;
const int maxN = 2e5, SIZE = 4*maxN;
const int INF = 0x3f3f3f3f;

int N, Q, a[maxN+1];
int lo[SIZE], hi[SIZE], mn[SIZE], mx[SIZE];

unordered_map>> prv;
unordered_map> nxt;
unordered_map segtree_idx;

void pull(int i){
    mn[i] = min(mn[2*i], mn[2*i+1]);
    mx[i] = max(mx[2*i], mx[2*i+1]);
}

void init(int i, int l, int r){
    lo[i] = l; hi[i] = r;
    if(l == r){
        int v = a[l];
        mn[i] = nxt[v].upper_bound(l) == nxt[v].end() ? INF : *nxt[v].upper_bound(l);
        mx[i] = prv[v].upper_bound(l) == prv[v].end() ? -INF : *prv[v].upper_bound(l);
        segtree_idx[l] = i;
        return;
    }

    int m = l+(r-l)/2;
    init(2*i, l, m);
    init(2*i+1, m+1, r);
    pull(i);
}

void assign(int i, int _mn, int _mx){
    i = segtree_idx[i];
    if(_mn != -1) mn[i] = _mn;
    if(_mx != -1) mx[i] = _mx;
    while(i > 1){
        i >>= 1;
    }
}

```

```

        pull(i);
    }
}

pii query(int i, int l, int r){
    if(l > hi[i] || r < lo[i])    return {INF, -INF};
    if(l <= lo[i] && hi[i] <= r)    return {mn[i], mx[i]};

    pii lmin = query(2*i, l, r);
    pii rmin = query(2*i+1, l, r);

    return {min(lmin.first, rmin.first), max(lmin.second, rmin.second)};
}

void update(int pos, int val){
    int old_val = a[pos];
    int old_next = nxt[old_val].upper_bound(pos) == nxt[old_val].end() ? INF : *nxt[old_val].upper_bound(pos);
    int old_prev = prv[old_val].upper_bound(pos) == prv[old_val].end() ? -INF : *prv[old_val].upper_bound(pos);

    prv[old_val].erase(pos);
    nxt[old_val].erase(pos);
    if(old_next != INF)    assign(old_next, -1, old_prev);
    if(old_prev != -INF)    assign(old_prev, old_next, -1);

    a[pos] = val;
    int new_next = nxt[val].upper_bound(pos) == nxt[val].end() ? INF : *nxt[val].upper_bound(pos);
    int new_prev = prv[val].upper_bound(pos) == prv[val].end() ? -INF : *prv[val].upper_bound(pos);
    prv[val].insert(pos);
    nxt[val].insert(pos);

    assign(pos, new_next, new_prev);
    if(new_next != INF)    assign(new_next, -1, pos);
    if(new_prev != -INF)    assign(new_prev, pos, -1);
}

int main(){
    scanf("%d %d", &N, &Q);

    segtree_idx.reserve(N);
    prv.reserve(N + Q);
    nxt.reserve(N + Q);

    for(int i = 1; i <= N; i++){
        scanf("%d", &a[i]);
        prv[a[i]].insert(i);
        nxt[a[i]].insert(i);
    }

    init(1, 1, N);
    for(int q = 0, k, x, y; q < Q; q++){
        scanf("%d %d %d", &k, &x, &y);
        if(k == 1)    update(x, y);
        else {
            pii res = query(1, x, y);
            int mn_val = res.first, mx_val = res.second;
            printf("%s\n", (mn_val <= y || mx_val >= x) ? "NO" : "YES");
        }
    }
}

```

Increasing Array Queries

```

#include

using namespace std;
typedef long long ll;
typedef pair pii;
const int maxN = 2e5+1;
const int SIZE = 4*maxN;

int N, Q, lo[SIZE], hi[SIZE];
ll a[maxN], ass[SIZE], mx[SIZE], sm[SIZE], ans[maxN];
vector queries[maxN];

int len(int i){
    return hi[i]-lo[i]+1;
}

void assign(int i, ll val){
    ass[i] = mx[i] = val;
    sm[i] = val * len(i);
}

void push(int i){
    if(ass[i]){
        assign(2*i, ass[i]);
        assign(2*i+1, ass[i]);
        ass[i] = 0;
    }
}

void pull(int i){
    sm[i] = sm[2*i] + sm[2*i+1];
    mx[i] = max(mx[2*i], mx[2*i+1]);
}

void init(int i, int l, int r){
    lo[i] = l; hi[i] = r;
    if(l == r){
        sm[i] = a[l];
        return;
    }
    int m = (l+r)/2;
    init(2*i, l, m);
    init(2*i+1, m+1, r);
    pull(i);
}

void update(int i, int l, int r, ll val){
    if(l > hi[i] || r < lo[i]) return;
    if(l <= lo[i] && hi[i] <= r){
        assign(i, val);
        return;
    }

    push(i);
    update(2*i, l, r, val);
    update(2*i+1, l, r, val);
    pull(i);
}

int orderOf(int i, int l, int val){
    if(lo[i] == hi[i]) return lo[i];
    push(i);
    int idx = -1;
    if(hi[2*i] <= l || mx[2*i] < val) idx = orderOf(2*i+1, l, val);
    else idx = orderOf(2*i, l, val);
    pull(i);
    return idx;
}

```

```

ll sum(int i, int l, int r){
    if(l > hi[i] || r < lo[i])    return 0;
    if(l <= lo[i] && hi[i] <= r)    return sm[i];

    push(i);
    ll left = sum(2*i, l, r);
    ll right = sum(2*i+1, l, r);
    pull(i);

    return left+right;
}

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 1; i <= N; i++){
        scanf("%lld", &a[i]);

    init(1, 1, N);
    for(int q = 0, l, r; q < Q; q++){
        scanf("%d %d", &l, &r);
        queries[l].push_back({r, q});
    }

    for(int i = 2; i <= N; i++) a[i] += a[i-1];
    for(int l = N; l >= 1; l--){
        int val = a[l]-a[l-1];
        int modifyR = (mx[l] < val ? N+1 : orderOf(1, l, val));
        update(1, l, modifyR-1, val);
        for(pii q : queries[l]){
            int r = q.first;
            int id = q.second;
            ans[id] = sum(1, l, r) - (a[r]-a[l-1]);
        }
    }

    for(int i = 0; i < Q; i++)
        printf("%lld\n", ans[i]);
}

```

Movie Festival Queries

```

#include

using namespace std;
const int logN = 17, maxT = 1e6;

int N, Q, dp[maxT+1][logN+1];
vector movies[maxT+1];

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 0, a, b; i < N; i++){
        scanf("%d %d", &a, &b);
        movies[a].push_back(b);
    }

    priority_queue, greater> ends;
    for(int t = maxT; t >= 0; t--){
        for(int b : movies[t])
            ends.push(b);
    }
}

```

```

    dp[t][0] = ends.empty() ? maxT+1 : ends.top();
}
for(int k = 1; k <= logN; k++){
    for(int t = 0; t <= maxT; t++){
        dp[t][k] = dp[t][k-1] > maxT ? maxT+1 : dp[dp[t][k-1]][k-1];

    }

    for(int q = 0, a, b; q < Q; q++){
        scanf("%d %d", &a, &b);
        int ans = 0;
        for(int k = logN; k >= 0; k--){
            if(dp[a][k] <= b){
                a = dp[a][k];
                ans += (1<

```

Forest Queries II

```

#include

using namespace std;
typedef long long ll;
const int maxN = 1000;

int N, Q;
char c[maxN+1][maxN+1];
ll ds[maxN+1][maxN+1];

void update(int x, int y, ll val){
    for(int i = x; i <= N; i += -i&i)
        for(int j = y; j <= N; j += -j&j)
            ds[i][j] += val;
}

ll query(int x, int y){
    ll sum = 0;
    for(int i = x; i > 0; i -= -i&i)
        for(int j = y; j > 0; j -= -j&j)
            sum += ds[i][j];
    return sum;
}

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 1; i <= N; i++){
        for(int j = 1; j <= N; j++){
            scanf(" %c", &c[i][j]);
            if(c[i][j] == '*')
                update(i, j, 1);
        }
    }
    for(int q = 0, t, x1, y1, x2, y2; q < Q; q++){
        scanf("%d %d %d", &t, &x1, &y1);
        if(t == 1){
            if(c[x1][y1] == '*'){
                update(x1, y1, -1);
                c[x1][y1] = '.';
            } else if(c[x1][y1] == '.'){
                update(x1, y1, 1);
                c[x1][y1] = '*';
            }
        } else if(t == 2){
            scanf("%d %d", &x2, &y2);

```

```

        printf("%lld\n", query(x2, y2)-query(x2,y1-1)-query(x1-1,y2)+query(x1-1,y1-1));
    }
}

```

Range Updates and Sums

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5;

int N, Q, t, a, b, lo[4*maxN], hi[4*maxN];
ll x, del[4*maxN], ass[4*maxN], sum[4*maxN];

int len(int i){
    return hi[i]-lo[i]+1;
}

void increment(int i, ll v){
    del[i] += v;
    sum[i] += len(i) * v;
}

void assign(int i, ll v){
    ass[i] = v;
    del[i] = 0;
    sum[i] = len(i) * v;
}

void push(int i){
    if(ass[i]){
        assign(2*i, ass[i]);
        assign(2*i+1, ass[i]);
        ass[i] = 0;
    }
    if(del[i]){
        increment(2*i, del[i]);
        increment(2*i+1, del[i]);
        del[i] = 0;
    }
}

void pull(int i){
    sum[i] = sum[2*i] + sum[2*i+1];
}

void build(int i, int l, int r){
    lo[i] = l; hi[i] = r;
    if(l == r){
        scanf("%lld", &sum[i]);
        return;
    }
    int m = l+(r-l)/2;
    build(2*i, l, m);
    build(2*i+1, m+1, r);
    pull(i);
}

void increment(int i, int l, int r, ll v){

```

```

    if(l > hi[i] || r < lo[i]) return;
    if(l <= lo[i] && hi[i] <= r){
        increment(i, v); return;
    }

    push(i);
    increment(2*i, l, r, v);
    increment(2*i+1, l, r, v);
    pull(i);
}

void assign(int i, int l, int r, ll v){
    if(l > hi[i] || r < lo[i]) return;
    if(l <= lo[i] && hi[i] <= r){
        assign(i, v); return;
    }

    push(i);
    assign(2*i, l, r, v);
    assign(2*i+1, l, r, v);
    pull(i);
}

ll query(int i, int l, int r){
    if(l > hi[i] || r < lo[i]) return 0;
    if(l <= lo[i] && hi[i] <= r){
        return sum[i];
    }

    push(i);
    ll lsum = query(2*i, l, r);
    ll rsum = query(2*i+1, l, r);
    pull(i);

    return lsum + rsum;
}

int main(){
    scanf("%d %d", &N, &Q);

    build(1, 1, N);
    for(int q = 0; q < Q; q++){
        scanf("%d %d %d", &t, &a, &b);
        if(t == 1){
            scanf("%lld", &x);
            increment(1, a, b, x);
        } else if(t == 2){
            scanf("%lld", &x);
            assign(1, a, b, x);
        } else if(t == 3){
            printf("%lld\n", query(1, a, b));
        }
    }
}

```

Polynomial Queries

```

#include

using namespace std;
typedef long long ll;

```

```

const int SIZE = 8e5;

int N, Q, t, a, b, lo[SIZE], hi[SIZE];
ll d1[SIZE], d2[SIZE], s1[SIZE], s2[SIZE];

int len(int i){
    return hi[i]-lo[i]+1;
}

ll sigma(int i){
    return ((ll) hi[i] * (hi[i]+1))/2 - ((ll) lo[i] * (lo[i]-1))/2;
}

void push(int i){
    if(d1[i]){
        d1[2*i] += d1[i];
        d1[2*i+1] += d1[i];
        d1[i] = 0;
    }
    if(d2[i]){
        d2[2*i] += d2[i];
        d2[2*i+1] += d2[i];
        d2[i] = 0;
    }
}

void pull(int i){
    ll l1 = s1[2*i] + (d1[2*i] * len(2*i));
    ll r1 = s1[2*i+1] + (d1[2*i+1] * len(2*i+1));
    s1[i] = l1 + r1;
    ll l2 = s2[2*i] + (d2[2*i] * sigma(2*i));
    ll r2 = s2[2*i+1] + (d2[2*i+1] * sigma(2*i+1));
    s2[i] = l2 + r2;
}

void build(int i, int l, int r){
    lo[i] = l; hi[i] = r;
    if(l == r){
        scanf("%lld", &s1[i]);
        return;
    }
    int m = l+(r-1)/2;
    build(2*i, l, m);
    build(2*i+1, m+1, r);
    pull(i);
}

void increment(int type, int i, int l, int r, ll v){
    if(l > hi[i] || r < lo[i]) return;
    if(l <= lo[i] && hi[i] <= r){
        if(type == 1) d1[i] += v;
        if(type == 2) d2[i] += v;
        return;
    }

    push(i);
    increment(type, 2*i, l, r, v);
    increment(type, 2*i+1, l, r, v);
    pull(i);
}

ll sum(int type, int i, int l, int r){
    if(l > hi[i] || r < lo[i]) return 0;
    if(l <= lo[i] && hi[i] <= r){
        if(type == 1) return s1[i] + (d1[i] * len(i));
        if(type == 2) return s2[i] + (d2[i] * sigma(i));
        return 0;
    }

    push(i);

```

```

    ll lsum = sum(type, 2*i, l, r);
    ll rsum = sum(type, 2*i+1, l, r);
    pull(i);

    return lsum + rsum;
}

void update(int l, int r){
    increment(1, 1, l, r, -l+1);
    increment(2, 1, l, r, 1);
}

ll query(int l, int r){
    return sum(1, 1, l, r) + sum(2, 1, l, r);
}

int main(){
    scanf("%d %d", &N, &Q);

    build(1, 1, N);
    for(int q = 0; q < Q; q++){
        scanf("%d %d %d", &t, &a, &b);
        if(t == 1) update(a, b);
        else if(t == 2) printf("%lld\n", query(a, b));
    }
}

```

Range Queries and Copies

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+1;

struct Node {
    Node *l, *r;
    int lo, hi;
    ll sum;
    Node(int low, int high, int val) : l(nullptr), r(nullptr), lo(low), hi(high), sum(val) {}
    Node(Node *left, Node *right) : l(left), r(right), sum(0) {
        if(left){
            sum += left->sum;
            lo = left->lo;
        }
        if(right){
            sum += right->sum;
            hi = right->hi;
        }
    }
};

int N, Q, type, k, a, b, x, cnt, t[maxN];
Node* seg[maxN];

Node* build(int l, int r){
    if(l == r) return new Node(l, r, t[l]);
    int m = (l+r)/2;
    return new Node(build(l, m), build(m+1, r));
}

```

```

Node* update(Node* node, int idx, int val){
    if(node->lo == node->hi)    return new Node(idx, idx, val);
    int m = (node->lo+node->hi)/2;
    if(idx <= m)    return new Node(update(node->l, idx, val), node->r);
    else    return new Node(node->l, update(node->r, idx, val));
}

ll query(Node* node, int l, int r){
    if(l > node->hi || r < node->lo)    return 0;
    if(l <= node->lo && node->hi <= r)    return node->sum;
    return query(node->l, l, r) + query(node->r, l, r);
}

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 1; i <= N; i++){
        scanf("%d", &t[i]);

        cnt = 1;
        seg[cnt++] = build(1, N);
        for(int q = 0; q < Q; q++){
            scanf("%d %d", &type, &k);
            if(type == 1){
                scanf("%d %d", &a, &x);
                seg[k] = update(seg[k], a, x);
            } else if(type == 2){
                scanf("%d %d", &a, &b);
                printf("%lld\n", query(seg[k], a, b));
            } else if(type == 3){
                seg[cnt++] = new Node(seg[k]->l, seg[k]->r);
            }
        }
    }
}

```

Missing Coin Sum Queries

```

#include

/* Fast Input and Output Template */
char _i[1<<24], _o[1<<10], __[20], _, _c; int _i0, _o0, _n;
#define readio()    { fread(_i, 1, 1<<24, stdin); }
#define writeio()    { fwrite(_o, 1, _o0, stdout); }
#define scanu(x)    { for (x = _i[_i0++] & 15; 47 < (_ = _i[_i0++]); x = x * 10 + (_ & 15)); }
#define putnumu(x)    { _ = 0; do __[_++] = x % 10 | '0'; while (x /= 10); while (_--) _o[_o0++] = __[_]; }
#define putnl()    { _o[_o0++] = '\n'; }
/* End of Template */

using namespace std;
typedef long long ll;
const int maxN = 2e5+2;
const int logC = 32, logN = 18;
const int INF = 0x3f3f3f3f;

int N, Q, mn[logC][logN][maxN];
ll pre[logC][maxN];

int layer(int x){
    return 31 - __builtin_clz(x);
}

ll sum(int l, int a, int b){

```

```
    return pre[l][b] - pre[l][a-1];  
}  
  
int minimum(int l, int a, int b){  
    a--; b--;  
    int len = b-a+1;  
    int k = (int) floor(log2(len));  
    return min(mn[l][k][a], mn[l][k][b-(1<= (1<
```
